

# Milestone 3: Critical Evaluation and Transfer Loop Unrolling and Controller Delay in High-Level Synthesis

Christian Percival  
Hochschule Hamm-Lippstadt  
Electronic Engineering

## I. MILESTONE 3: CRITICAL EVALUATION AND TRANSFER

### A. *Strengths of the Approach*

- The paper identifies an important HLS problem that can easily be overlooked.
- It shows that loop unrolling affects not only latency, but also controller delay.
- It connects a high-level compiler optimization to low-level hardware timing effects.
- The approach avoids exhaustive synthesis of all unroll factors.
- The estimation method can reduce exploration time significantly.
- The priority-based selection method gives a practical way to choose promising unroll factors.
- The paper supports the idea that HLS optimization must consider hardware constraints.

### B. *Limitations of the Approach*

- The delay model depends on the synthesis flow and target technology.
- The original experiments use a specific behavioral synthesis framework and ASIC library.
- The exact delay constants may not apply directly to FPGA-based HLS tools.
- Modern HLS tools may optimize the design differently.
- Controller delay may be difficult to isolate in modern synthesis reports.
- The method focuses mainly on controller delay, while real timing also includes datapath, routing, memory, and placement effects.
- The approach may be less accurate for designs with complex memory behavior or tool-specific optimizations.

### C. *Hidden or Restrictive Assumptions*

- The loop structure must be suitable for unrolling.
- Loop bounds should be known or predictable.
- The effect of unrolling should be estimable from high-level information.
- The controller delay model must be calibrated for the target synthesis environment.
- The approach assumes that delay estimation is accurate enough to guide design-space exploration.

- It also assumes that reducing exploration time is more important than finding an absolutely optimal result through exhaustive synthesis.

### D. *Risks*

- An inaccurate delay estimate could select an unroll factor that fails timing.
- A conservative estimate could reject useful unroll factors.
- Modern HLS tools may hide controller details, making validation difficult.
- A student implementation may only measure total timing and resource usage, not pure controller delay.
- If the benchmark is too simple, the controller-delay effect may not be very visible.
- If the benchmark is too complex, the result may become difficult to explain.

### E. *Suitable Application Examples*

- The approach is suitable for loop-based hardware accelerators.
- Suitable examples include:
  - dot-product accelerators,
  - FIR filters,
  - simple image-processing kernels,
  - matrix or vector operations,
  - embedded signal-processing loops.
- These applications often contain repeated loop operations and can benefit from partial unrolling.
- The approach is especially useful when there is a strict timing budget.

### F. *Unsuitable or Problematic Application Examples*

- The approach may be less suitable for code with highly irregular control flow.
- It may be less useful when loop bounds are unknown or data-dependent.
- It may be difficult to apply when memory access patterns dominate performance.
- It may be less effective if the HLS tool already performs aggressive automatic optimization.
- It may also be less useful for very small loops where exhaustive exploration is already cheap.

### G. Transfer to a Student Implementation

- The original ASIC-based synthesis setup is likely not available for a low-budget student project.
- A practical student implementation can instead use:
  - a Python-based estimation model,
  - AMD Vitis HLS,
  - Intel HLS,
  - Bambu HLS,
  - or another available university HLS tool.
- The implementation should not try to reproduce the original paper exactly.
- The goal should be to demonstrate the same engineering principle.
- The student experiment can compare different unroll factors for a small benchmark loop.
- Example unroll factors:
  - 1,
  - 2,
  - 4,
  - 8,
  - 16,
  - 32.
- For each unroll factor, the following should be recorded:
  - latency in cycles,
  - estimated clock period,
  - timing slack,
  - LUT or ALM usage,
  - flip-flop or register usage,
  - DSP usage,
  - BRAM or memory usage,
  - synthesis time if available.

### H. Possible Python Simulation

- A Python model can be used if HLS tools are not available.
- The Python model can estimate:
  - operation count,
  - number of loop iterations after unrolling,
  - estimated latency,
  - estimated number of states,
  - estimated state bits,
  - estimated controller delay.
- The simulation would not prove real hardware timing.
- However, it can help demonstrate the trend that unrolling reduces loop iterations while increasing controller complexity.

### I. Possible HLS Experiment

- A small dot-product function can be written in C or C++.
- The loop can be synthesized multiple times with different unroll factors.
- The results can be compared using the HLS synthesis reports.
- The most important output would be a table and graph showing:

- unroll factor versus latency,
  - unroll factor versus resource usage,
  - unroll factor versus clock estimate or timing slack.
- This would provide a practical demonstration of the paper's main idea.

### J. Possible Extensions

- Compare loop unrolling with loop pipelining.
- Study how memory partitioning affects the benefit of loop unrolling.
- Test more than one benchmark loop.
- Compare full unrolling with partial unrolling.
- Extend the model to include datapath delay and routing delay.
- Investigate how modern FPGA HLS tools report controller-related timing.

### K. Critical Conclusion

- The paper provides a useful reminder that HLS optimizations must be evaluated from a hardware perspective.
- Loop unrolling can reduce latency, but it can also increase controller complexity and timing pressure.
- The best unroll factor is usually a tradeoff, not simply the largest possible factor.
- Estimation-based exploration is useful because exhaustive synthesis can be too slow.
- For a student project, the most realistic implementation is a simplified estimation model supported by a small HLS experiment if tools are available.
- The final seminar paper should clearly distinguish between:
  - what the original paper proves,
  - what can be reproduced in a student environment,
  - and what remains a limitation or open question.

### REFERENCES

- [1] S. Kurra, N. K. Singh, and P. R. Panda, "The Impact of Loop Unrolling on Controller Delay in High Level Synthesis," DATE, 2007.